

Current Recursion and Tensor-Network Execution in PYAMPLICOL

Purpose. This note summarizes the matrix-element generation strategy used by AMPLICOL and the corresponding PYAMPLICOL implementation. The common idea is to avoid explicit Feynman-diagram enumeration. Instead, matrix elements are assembled from a directed acyclic graph (DAG) of off-shell currents. AMPLICOL emits Fortran code that numerically sweeps this current table. PYAMPLICOL builds the same logical DAG in Python, lowers bounded pieces to tensor-network expressions handled by Spenso, compiles the resulting parametric scalar/vector expressions with Symbolica evaluators, and can now execute the staged sweep from Rust through the PyO3 runtime `rusticol`.

Current status. The generic LC/NLC/full-colour staged-DAG implementation is the production PYAMPLICOL workflow. Process generation writes schema-v2 artifacts loaded by `rusticol`; model parameters are runtime inputs and may be overridden from TOML. The default benchmark route is Rusticol with SymJIT stage evaluators, stage-local inputs, output chunk size 128, batch size 64, ten Horner iterations, Symbolica’s default CPE choice, and enlarged Horner/common-pair limits. The production matrix JIT row uses SymJIT O1 for generation-speed comparisons; dedicated Z-family scans also show O3/ASM/C++ variants. The result matrices are generated as separate $\text{\LaTeX}$ inputs, and the benchmark writers refresh `pyAmpliCol.pdf` after each completed cell.

1 The AmpliCol Current Recursion

For a fixed leading-color ordering, a current is labelled by the external subset it contains, the particle type carried by the off-shell line, and any spin/chirality information. Schematically, if I is a set or ordered interval of external colored legs, a current is computed as

$$\mathcal{J}_{t,\chi}^\alpha(I) = P^\alpha{}_\beta(t, p_I) \sum_{\substack{I=A \cup B \\ A \cap B = \emptyset}} \mathcal{V}_{\gamma\delta}^\beta(t_A, t_B \rightarrow t) \mathcal{J}_{t_A, \chi_A}^\gamma(A) \mathcal{J}_{t_B, \chi_B}^\delta(B). \quad (1)$$

Here t is the current type (gluon, quark, auxiliary tensor, etc.), χ is the Weyl chirality where relevant, $p_I = \sum_{i \in I} p_i$, P is the propagator, and $\mathcal{V}$ is the appropriate Feynman-rule tensor. The four-gluon vertex is treated through the auxiliary antisymmetric tensor route, so it is still represented by cubic current combinations.

AMPLICOL’s optimized mode does not loop over full helicity configurations as independent computations. It weaves helicity information into the current table itself. Each external helicity state is a source current with a unique bit in an `ext_cur`-style bitset. An internal current is identified by

$$K(\mathcal{J}) = (t, \chi, \text{bin}(I), \text{ext_cur}), \quad (2)$$

where `bin(I)` records the external subset and `ext_cur` is the union of the source-current bits feeding the current. When a new vertex contribution produces the same key as an existing current, AMPLICOL appends that vertex to the existing current rather than creating a duplicate node. After all currents are constructed, `curr2amp` records the two endpoint currents whose contraction gives each retained helicity amplitude.

The generated Fortran library has the same layered structure:

```

subroutine evaluate_amp(p, amps)
  complex(kind=8) :: val_c(1:6, n_cur)
  complex(kind=8) :: int_c(1:6, n_vert)

  call compute_external_currents(pp, val_c)
  do size = 2, n_external_minus_one
    call compute_vertices_size(pp, val_c, int_c)
    call compute_currents_size(pp, val_c, int_c)
  end do
  call compute_amps(amps, val_c)
end subroutine

```

The arrays `val_c` and `int_c` are the numerical current and interaction tables. This is the performance-critical idea that PYAMPLICOL keeps: one topological sweep produces all retained amplitudes, and shared parent currents are evaluated once.

2 The pyAmpliCol DAG

PYAMPLICOL first builds an explicit Python representation of the same current table. In the generic compiler, a current is identified by a model-driven physics index rather than by a process-family name. The identity used for deduplication is

$$K_{\text{PYAMPLICOL}} = (\text{particle_id}, L, H, \chi, S, F, Q, C, P, A), \quad (3)$$

where L is the sorted tuple of external labels, H is the AMPLICOL-style helicity-source ancestry bitset, χ is chirality, S is any spin-state tag needed by the source or propagator, F is flavour flow, Q is charge flow, C is the leading-color state, P is the momentum mask, and A is an optional auxiliary kind such as the antisymmetric tensor used for the four-gluon vertex. The implementation also carries `ordered_external_labels`. That field is leading-color ordering metadata used by the color engine and amplitude grouping, but it is deliberately not part of equality or hashing. Deduplication is therefore exactly equality of Eq. (3); no rule is allowed to ask whether the process is “ Z plus jets” or any other family.

field	role in the current identity
<code>particle_id</code>	Model/UFO particle id, e.g. 21, 23, 1, -1 , or an auxiliary id.
<code>external_labels</code> and <code>external_mask</code>	Which external legs are contained in the current. The mask is the bit representation of the labels.
<code>helicity_ancestry</code>	Bitwise union of the source-current helicity bits feeding this current. This is how helicity configurations are woven into one shared DAG.
<code>chirality</code> and <code>spin_state</code>	Local spin information, especially for Weyl fermions and massive-vector sources.
<code>flavour_flow</code> and <code>charge_flow</code>	Quantum-number flow checked locally by model vertices, e.g. $u \rightarrow dW^+$ or $e^+e^- \rightarrow \gamma/Z$.
<code>color_state</code>	Colour-ordered open-line or trace state used to build the amplitude vector. LC, NLC and full colour differ in the final sparse colour contraction over this vector.
<code>momentum_mask</code>	Which all-outgoing momenta are summed for propagators and momentum-dependent vertices.
<code>auxiliary_kind</code>	Optional tag distinguishing auxiliary tensor currents from ordinary particles.

Source currents are built from external leg metadata. Internal currents are then generated by increasing current size, with all interactions that feed the same current index attached to one node.

```

for external_helicity_source s:
    add_current(key=(pdg_s, labels_s, chirality_s, bit_s), is_source=True)

for current_size in increasing_sizes:
    for compatible_split A | B:
        result_key = combine_key(key[A], key[B], vertex_kind)
        result = add_or_reuse_current(result_key)
        add_interaction(left=A, right=B, result=result, vertex=vertex_kind)

for retained_helicity_ancestry:
    add_amplitude(left=current_with_Z_and_gluons, right=anti_current)

prune_dead_currents_after_helicity_filter()

```

At runtime, PYAMPLICOL mirrors the Fortran table layout with contiguous arrays:

$$C_{s,i,c} \equiv \text{current_rows}[s, i, c], \quad T_{s,v,c} \equiv \text{interaction_rows}[s, v, c],$$

where s is the phase-space sample in the current batch, i is a current id, v is an interaction id, and c is a component index. External wavefunctions fill source-current rows once per sample. Bounded evaluator blocks then fill interactions and derived currents stage by stage:

```

fill_source_currents(current_rows, momenta, helicity_sources)

for stage in current_size_layers:
    state = ParamBuilder.pack(current_rows, interaction_rows, momenta)
    interaction_rows[:, stage.vertices] = vertex_eval[stage](state)
    current_rows[:, stage.currents] = current_eval[stage](state)

amps = amplitude_eval(ParamBuilder.pack(current_rows, momenta))
me2 = normalization * sum(weight[h] * abs(amps[h])**2 for h in retained_h)

```

This is why the evaluator-only benchmark is meaningful: the expensive recursion is inside compiled evaluator calls, while Python only performs source wavefunction construction, parameter packing, and table transfers.

3 Spenso Tensor Networks and hep_lib

Spenso is used to express Lorentz, spinor, auxiliary-tensor, and eventually color contractions in a representation-aware way. A tensor-network expression contains named tensors with typed slots:

$$\mathcal{V}_{\mu\nu}{}^\rho(p_A, p_B) A^\mu B^\nu, \quad \text{or in code: } V(m("a"), m("b"), m("out")) * A(m("a")) * B(m("b")).$$

The expression is not expanded. It is a compact product of tensor factors with shared representation labels. The library `hep_lib` is the dictionary that tells Spenso what each tensor name means: dense model tensors, metric tensors, momentum-dependent propagators, and parametric rank-one tensors representing runtime inputs.

```
library = model.build_tensor_library()      # returns TensorLibrary.hep_lib_atom()
builder = ParamBuilder()
mink = Representation.mink(4)

builder.register_rank1_tensor(
    library,
    tensor_name="current_A",
    representation=mink,
    head=("stage", "A"),
    length=4,
    role="block_current",
)
builder.register_rank1_tensor(
    library,
    tensor_name="p_A",
    representation=mink,
    head=("stage", "pA"),
    length=4,
    role="block_momentum",
    real_valued=True,
)

raw = V3g(mink("a"), mink("b"), mink("out"), p_A="p_A", p_B="p_B") \
    * TensorName("current_A")(mink("a")).to_expression() \
    * TensorName("current_B")(mink("b")).to_expression()

network = TensorNetwork(raw, library)
network.execute(library=library)
out_tensor = network.result_tensor(library)
evaluator = out_tensor.evaluator(params=builder.parameter_symbols())
```

The important point is that `hep_lib` stays fixed during the execution. New graph currents are not registered as permanent physics tensors. Instead, the current values are runtime parameters for a bounded block. Spenso executes the factorized tensor network, contracts the representation indices, and returns a parametric output tensor. Symbolica then compiles that parametric tensor to an evaluator. For high multiplicity, PYAMPLICOL interleaves this process: after a current or current-size layer is contracted, its numerical output is written back into the current table and becomes an input parameter for later blocks. This avoids one giant scalar expression and keeps the DAG close to AMPLICOL's execution model.

The current NLC/full-colour implementation keeps the kinematic DAG colour-ordered and applies a sparse colour matrix to the amplitude vector. For open-line sectors, including arbitrary many quark lines with gluons distributed on the lines, PYAMPLICOL contracts the product of left colour strings with the conjugated right strings into closed fundamental traces. Those small colour trace expressions are simplified separately from the kinematic DAG; NLC is obtained by keeping the leading and first $1/N_c^2$ -suppressed powers, while full colour keeps the complete sparse metric. If a later implementation puts explicit colour tensors inside a Spenso block, colour simplification still belongs before Spenso execution:

```
raw ← idenso.simplify_color(raw),      then build TensorNetwork(raw, hep_lib).
```

For the present NLC/full-colour milestone, colour algebra is confined to the small colour-basis overlap problem and the Rusticol final contraction, not to expanded kinematic expressions. Fortran AmpliCol is the direct reference for zero, one and two quark-line colour matrices; beyond that, pyAmpliCol's generic open-line metric is validated through raw-amplitude probes and internal colour consistency checks.

4 Example: $d\bar{d} \rightarrow Zgg$

This section spells out one concrete DAG so that the split between compiled current contractions and Rust-interpreted scheduling is explicit. The user-level process is

$$d(p_1) \bar{d}(p_2) \rightarrow Z(p_3) g(p_4) g(p_5).$$

PYAMPLICOL stores the process in the all-outgoing convention. Thus labels $1, \dots, 5$ carry the fields

$$\bar{d}_1, \quad d_2, \quad Z_3, \quad g_4, \quad g_5,$$

with momenta $-p_1, -p_2, p_3, p_4$, and p_5 , respectively. A label set such as $\{2, 4, 5\}$ is represented by $L = (2, 4, 5)$ and by the mask

$$m(2, 4, 5) = 2^{2-1} + 2^{4-1} + 2^{5-1} = 26.$$

The momentum mask is the same bit pattern in this example, and it tells Rusticol which stored momentum sum must be passed to the next compiled evaluator.

For one leading-color sector, the colored ordered labels are

$$O = (2, 4, 5, 1),$$

with the color-singlet Z_3 allowed to attach anywhere on the quark line. The color engine may also provide the sector $O = (2, 5, 4, 1)$. Nothing in the DAG builder special-cases this as a “ $Z + 2g$ ” process: source currents are ordinary external-leg currents and all later currents are discovered from model vertices.

Source current indices

For a fixed retained source helicity choice, the source entries look as follows. The bit b_i denotes the unique helicity-source bit assigned to that external state.

source	current	mask	index payload
S_1	$\mathcal{J}_{\bar{d}}(\{1\})$	1	(pid = -1, $L = (1)$, $H = b_1, \chi, S, F =$ $(-1), Q, C_{\bar{q}}, P = 1)$
S_2	$\mathcal{J}_d(\{2\})$	2	(pid = 1, $L = (2)$, $H = b_2, \chi, S, F =$ $(1), Q, C_q, P = 2)$
S_3	$\epsilon_Z(\{3\})$	4	(pid = 23, $L = (3)$, $H = b_3, S =$ $h_Z, C_{\text{singlet}}, P = 4)$
S_4	$\epsilon_g(\{4\})$	8	(pid = 21, $L = (4)$, $H = b_4, S = h_4, C_g, P = 8)$
S_5	$\epsilon_g(\{5\})$	16	(pid = 21, $L = (5)$, $H = b_5, S = h_5, C_g, P = 16)$

The actual stored index also contains the exact leading-color sector id, line-group metadata, and the current dimension. For this example the quark and anti-quark source currents have Weyl-spinor components, gluons and the Z have Lorentz-vector components, and auxiliary tensor currents have the antisymmetric tensor components required by the cubicized four-gluon rule.

What each compiled stage computes

The generic stage compiler groups interactions by the size of the result current. For each group it emits a multi-output Symbolica evaluator. That evaluator is compiled once and receives only parent-current components, momentum sums, and real couplings as parameters. It returns the components of all result currents in the stage.

For the sector $O = (2, 4, 5, 1)$, the schematic stage contents are:

$$E_2 : \quad \mathcal{J}_g(4, 5) = P_g(p_{45}) \mathcal{V}_{3g}(S_4, S_5; p_4, p_5), \quad (4)$$

$$\mathcal{J}_T(4, 5) = \mathcal{V}_{ggT}(S_4, S_5), \quad (5)$$

$$\mathcal{J}_d(2, 4) = P_q(p_{24}) \mathcal{V}_{ggq}(S_2, S_4; p_2, p_4), \quad (6)$$

$$\mathcal{J}_d(2, 3) = P_q(p_{23}) \mathcal{V}_{qZq}(S_2, S_3; p_2, p_3), \quad (7)$$

plus the analogous allowed currents for the other color sector and for the anti-quark side. The notation E_2 means a compiled evaluator for all two-label result currents, not a Python function that loops over these formulas component by component.

The next stage consumes the rows written by E_2 :

$$E_3 : \quad \mathcal{J}_d(2, 4, 5) = P_q(p_{245}) [\mathcal{V}_{ggq}(\mathcal{J}_d(2, 4), S_5) + \mathcal{V}_{ggq}(S_2, \mathcal{J}_g(4, 5))], \quad (8)$$

$$\mathcal{J}_d(2, 3, 4) = P_q(p_{234}) [\mathcal{V}_{qZq}(\mathcal{J}_d(2, 4), S_3) + \mathcal{V}_{ggq}(\mathcal{J}_d(2, 3), S_4)]. \quad (9)$$

The stage compiler does not expand these expressions into diagrams. It collects all interaction records whose output index is the same current and compiles the sum into the output components of that current. If two different splits produce the same current index, their contributions are accumulated inside the same compiled stage output.

The four-label stage then builds the endpoint current used by the closure:

$$E_4 : \quad \mathcal{J}_d(2, 3, 4, 5) = P_q(p_{2345}) \sum_{\text{compatible splits}} \mathcal{V}(\mathcal{J}_{\text{left}}, \mathcal{J}_{\text{right}}). \quad (10)$$

The sum includes, for example, gluon-first and Z -insertion contributions such as

$$\mathcal{V}_{qZq}(\mathcal{J}_d(2, 4, 5), S_3), \quad \mathcal{V}_{qgq}(\mathcal{J}_d(2, 3, 4), S_5), \quad \mathcal{V}_{qgq}(\mathcal{J}_d(2, 3), \mathcal{J}_g(4, 5)).$$

Those are still local vertex applications selected by the model and color engine. The result is one stored current row, identified by the full CurrentIndex and by its value-slot range in the artifact.

Finally, a compiled amplitude evaluator returns raw complex roots:

$$E_{\text{amp}} : \quad R_a = \mathcal{V}_{\text{close}}(\mathcal{J}_d(2, 3, 4, 5), S_1), \quad (11)$$

with one output for each retained root. Roots with the same $(H, \text{color sector}, O)$ coherent-group key are summed before squaring. The reported leading-color matrix element is

$$|\mathcal{M}|_{\text{LC}}^2 = \mathcal{N}_{\text{AC}} \sum_{\text{groups } g} w_g \left| \sum_{a \in g} R_a \right|^2. \quad (12)$$

The same example as a DAG

A stripped-down adjacency-list representation of this example is:

```
sources interpreted by Rusticol:
S1 = J(pid=-1, labels={1}, H=b1, color=qbar-line)
S2 = J(pid= 1, labels={2}, H=b2, color=q-line)
S3 = J(pid=23, labels={3}, H=b3, color=singlet)
S4 = J(pid=21, labels={4}, H=b4, color=line gluon)
S5 = J(pid=21, labels={5}, H=b5, color=line gluon)

compiled stage E2, subset_size=2:
I0: S4 + S5 -- V3g, Pg --> J(pid=21, labels={4,5})
I1: S4 + S5 -- VggT --> J(aux=T, labels={4,5})
I2: S2 + S4 -- Vqgq,Pq --> J(pid=1, labels={2,4})
I3: S2 + S3 -- VqZq,Pq --> J(pid=1, labels={2,3})

compiled stage E3, subset_size=3:
I4: J(2,4) + S5 -- Vqgq,Pq --> J(pid=1, labels={2,4,5})
I5: S2 + J(4,5) -- Vqgq,Pq --> J(pid=1, labels={2,4,5})
I6: J(2,4) + S3 -- VqZq,Pq --> J(pid=1, labels={2,3,4})
I7: J(2,3) + S4 -- Vqgq,Pq --> J(pid=1, labels={2,3,4})

compiled stage E4, subset_size=4:
I8: J(2,4,5) + S3 -- VqZq,Pq --> J(pid=1, labels={2,3,4,5})
I9: J(2,3,4) + S5 -- Vqgq,Pq --> J(pid=1, labels={2,3,4,5})
I10: J(2,3) + J(4,5) -- Vqgq,Pq --> J(pid=1, labels={2,3,4,5})

compiled amplitude stage:
A0: J(pid=1, labels={2,3,4,5}) + S1 -- close --> R0
```

This listing is schematic: the real artifact contains all retained helicity ancestries, both leading-color sectors, source dimensions, output component ranges, momentum-slot ids, couplings, and color weights. The important point is the division of labor. The evaluator E_s performs the tensor contractions and the algebraic sum over interactions in stage s . Rusticol interprets the DAG schedule: fill sources, compute momentum sums, call E_2 , scatter its outputs into current storage, call E_3 , scatter again, and so on until E_{amp} and the final coherent square-sum.

5 Rusticol: the Implemented Rust Runtime

The previous staged PYAMPLICOL runtime proved that the shared-current DAG could match Fortran AMPLICOL numerically, but its Python loop still moved current tables between many evaluator calls. The current implementation

keeps Python as the process generator and validation driver, but moves the hot eager-DAG sweep into a PyO3 extension named `rusticol`. The generated artifact is now a self-contained process directory used by both runtimes:

```
rt_py = pyamplicol.load_process(process_dir, runtime="python")
values_py = rt_py.evaluate(momenta)

import rusticol
rt = rusticol.Runtime.load(process_dir)
values = rt.evaluate(momenta)
profile = rt.profile(momenta)
```

The Python and Rust runtimes read the same `process_manifest.json`, the same Symbolica evaluator manifests, and the same validation momenta. The Python path is kept as a transparent reference implementation. The Rust path is the performance path.

A process directory contains four classes of data. First, it records process and model metadata: external PDG order, momentum conventions, particle and vertex tables, couplings, masses, symmetry factors, and leading-color normalization. Second, it stores the shared-current DAG: source currents, current ids, dimensions, interaction stages, output slots, retained amplitude records, helicity weights, and color factors. Third, it stores the compiled Symbolica evaluator artifacts for each stage and for the final amplitude/raw-sum stage. Fourth, it contains standalone validation assets: `validation_momenta.json` with decimal-string momenta and a `check_standalone.py` script that imports only `rusticol`, plus `numpy` when available for the optimized f64 input path.

```
# Generated by pyAmpliCol.
process_dir/
  process_manifest.json
  manifest.json
  validation_momenta.json
  check_standalone.py
  compiled/           # C++/ASM shared libraries when present
  ... evaluator states ...
```

`rusticol.Runtime.load` validates the schema, checks that the current process family is supported, loads every Symbolica evaluator, computes direct state offsets for stage outputs, allocates reusable scratch buffers, and builds a fixed execution plan. The optimized f64 path accepts NumPy arrays with shape `(batch, n_external, 4)`. For each batch, Rust fills source wavefunctions and momentum sums, executes stage evaluators in topological order, writes the returned current components into the contiguous state table, executes the amplitude/raw-sum evaluator, applies the Fortran-matched normalization, and returns a NumPy array of matrix elements.

```
for point in batch {
    fill_source_currents(state_row, point);
    fill_momentum_sums(state_row, point);
}

for stage in stages {
    evaluated = stage.evaluator.evaluate_batch(state);
    assign_stage_outputs(state, evaluated, stage.output_offsets);
}

raw = amplitude_stage.evaluate_raw_sums(state);
me2 = normalization * raw;
```

The `profile` API reports the same split used in the benchmarks: source fill, momentum setup, stage evaluator time, output assignment, amplitude evaluator time, final reduction, total wall time, and memory snapshots where the platform exposes them.

The precision API follows Symbolica’s model. `evaluate` and `evaluate_with_prec(..., 16)` use the specialized native f64 path. `evaluate_with_prec(..., 32)` maps the serialized exact evaluator state to Symbolica’s `DoubleFloat` coefficients. Other positive precisions map to arbitrary-precision `Float` evaluators. The high-precision routes share the same stage orchestration code through generic Rust traits, while the f64 route remains specialized and avoids high-precision allocation overhead.

The artifact schema is process-generic. The staged execution format no longer needs a Rust-side process-family branch: each process directory supplies the source layout, current slots, stage records, amplitude roots, coherent groups, and evaluator artifacts needed by `Rusticol`. If a model vertex or color rule is not implemented yet, generation fails with an explicit unsupported-kernel diagnostic rather than falling back to a hidden family path.

6 Shared LC Ordering Recycling

LC process generation now has two related modes. For ordinary integration benchmarks, PYAMPLICOL times one selected LC ordering per phase-space point. The selected artifact is pruned before Symbolica evaluator construction, so no dynamic colour-flow selector remains in those expressions. For all-flow studies, PYAMPLICOL writes an `lc-replay-partition-schema-v2` artifact: the root manifest owns several selected-sector representative sidecars, and each sidecar replays an exact block of sibling LC orderings by external-label permutation. The blocks are chosen so the incoming-label set is preserved. Thus the union covers the same colour orderings probed by AMPLICOL’s colour driver, without building the old sector-local all-ordering clone.

This is an exact partition, not a representative approximation. For $gg \rightarrow t\bar{t} + 5g$, for example, there are $7! = 5040$ LC orderings of the seven gluon labels. They split into 21 replay representatives, each covering $2!5! = 240$ orderings while keeping the two incoming gluon labels incoming. All generated representative outputs are kept on disk, so any block can be re-timed or inspected without regenerating the process.

Process	Orders	Reps	AMPLICOL gen	PYAMPLICOL gen	JIT	Run [s/pt]	Rel. diff.
$gg \rightarrow t\bar{t} + 3g$	120	10×12	6.55	8.34	3.04	0.00204 / 0.00345	8.3×10^{-13}
$gg \rightarrow t\bar{t} + 4g$	720	15×48	27.6	31.6	13.0	0.0358 / 0.0488	8.1×10^{-13}
$gg \rightarrow t\bar{t} + 5g$	5040	21×240	255	143	57.7	0.826 / 1.287	1.1×10^{-12}

Runtime entries are PYAMPLICOL/AMPLICOL helicity-summed all-flow wall time per phase-space point. The first two AMPLICOL generation entries come from the same generated-library refresh used by the LC matrix. For $gg \rightarrow t\bar{t} + 5g$, the measured helicity-summed all-flow workflow is already faster than AMPLICOL both in generation (143 s versus 255 s) and runtime (0.826 s per point versus 1.287 s per point). The process-matrix tables below use a different second runtime slot: all colour flows at one deterministic fixed source helicity, while the first runtime slot remains the selected colour flow with the usual helicity sum.

7 Generic LC Performance Matrix

Each cell compares generated-library AMPLiCOL leading-colour production against PYAMPLiCOL staged-DAG JIT O1 at the same final-state multiplicity n . Generation builds exact LC replay partitions covering all colour orderings; the PYAMPLiCOL selected-flow slot times the matched LC sector with the usual helicity sum, while the all-flow slot times the replay-partition aggregate for one deterministic fixed source helicity. The AMPLiCOL slot shows selected-flow/all-flow values separated by /; the all-flow AMPLiCOL timing uses the same fixed-helicity colour probe. PYAMPLiCOL runtime multipliers use wall time relative to the corresponding selected-flow or all-flow AMPLiCOL number. Generation entries are in seconds; AMPLiCOL runtime entries and absolute PYAMPLiCOL runtimes are in microseconds per point. If no direct AMPLiCOL reference exists, completed PYAMPLiCOL slots show absolute timings. Ratio colours are green below one, orange below two, and red otherwise. The summary rows give $\min|\max|_{\text{avg}}|\text{med}$; multiplier summaries add sum , the ratio of paired summed PYAMPLiCOL and AMPLiCOL times. For LC, the two runtime summary rows match the one-flow helicity-summed and all-flow fixed-helicity runtime slots. Missing and structural N/A cells are ignored. Validated rows use the same phase-space point and a 10^{-8} relative tolerance.

A red **VALIDATION FAILED** marker is shown next to any validated entry whose relative difference exceeds 10^{-8} .

ID	base process	n=1				n=2				n=3			
1	$d\bar{d} \rightarrow Z + (n-1)g$	2.465	/ 2.465	(x0.07)	(x0.07)	2.939	/ 2.939	(x0.093)	(x0.093)	2.146	/ 2.146	(x0.082)	(x0.082)
		0.0211	/ 0.95	(x10.1)	(x0.176)	0.128	/ 1.082	(x3.16)	(x0.207)	0.413	/ 1.18	(x1.93)	(x0.495)
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	2.893	/ 2.893	(x0.063)	(x0.063)	2.794	/ 2.794	(x0.085)	(x0.085)	2.105	/ 2.105	(x0.079)	(x0.079)
		0.0137	/ 0.982	(x13.2)	(x0.168)	0.0731	/ 1.008	(x4.30)	(x0.231)	0.246	/ 1.169	(x2.38)	(x0.511)
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A				2.777	/ 2.777	(x0.065)	(x0.065)	2.159	/ 2.159	(x0.075)	(x0.075)
						0.0654	/ 1.016	(x3.93)	(x0.201)	0.171	/ 1.114	(x2.42)	(x0.265)
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A				2.065	/ 2.065	(x0.076)	(x0.076)	2.1	/ 2.1	(x0.086)	(x0.086)
						0.0296	/ 0.986	(x6.31)	(x0.2)	0.0614	/ 1.067	(x4.67)	(x0.26)
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A				2.257	/ 2.257	(x0.069)	(x0.069)	2.214	/ 2.214	(x0.074)	(x0.074)
						0.158	/ 1.005	(x2.78)	(x0.226)	0.712	/ 1.066	(x1.53)	(x0.32)
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A				2.257	/ 2.257	(x0.083)	(x0.083)	2.46	/ 2.46	(x0.088)	(x0.088)
						0.103	/ 1.031	(x4.52)	(x1.69)	0.667	/ 1.493	(x1.82)	(x1.54)
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A				2.058	/ 2.058	(x0.092)	(x0.092)	2.198	/ 2.198	(x0.111)	(x0.111)
						0.0452	/ 1	(x6.11)	(x0.409)	0.244	/ 1.272	(x2.37)	(x0.992)
8	$gg \rightarrow gg + (n-2)g$	N/A				2.18	/ 2.18	(x0.073)	(x0.073)	2.22	/ 2.22	(x0.126)	(x0.126)
						0.161	/ 1.063	(x2.40)	(x0.308)	0.875	/ 2.51	(x1.43)	(x1.01)
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A				N/A				2.666	/ 2.666	(x0.067)	(x0.067)
										1.566	/ 1.284	(x1.45)	(x0.33)
10	$d\bar{d} \rightarrow e^+e^- ZH + (n-4)g$	N/A				N/A				N/A			
11	$d\bar{d} \rightarrow t\bar{t} ZH + (n-4)g$	N/A				N/A				N/A			
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A				N/A				N/A			
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A				N/A				N/A			
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A				N/A				N/A			
summary: gen		2.47	2.89	2.68	2.68	2.06	2.94	2.42	2.26	2.1	2.67	2.25	2.2
		x0.063	x0.07	x0.067	x0.067	x0.065	x0.093	x0.079	x0.08	x0.067	x0.126	x0.088	x0.082
summary: run one-flow, hel.		0.0137	0.0211	0.0174	0.0174	0.0296	0.161	0.0954	0.088	0.0614	1.57	0.551	0.413
sum		x10.1	x13.2	x11.6	x11.6	x2.40	x6.31	x4.19	x4.12	x1.43	x4.67	x2.22	x1.93

ID	base process					n=1					n=2					n=3				
summary: run all-flows, one-hel						0.95	0.982	0.966	0.966		0.986	1.08	1.02	1.01		1.07	2.51	1.35	1.18	
						x0.168	x0.176	x0.172	x0.172	x0.172	x0.2	x1.69	x0.434	x0.228	x0.435	x0.26	x1.54	x0.635	x0.495	x0.707

Generic LC Performance Matrix (continued)

ID	base process	n=4				n=5				n=6			
1	$d\bar{d} \rightarrow Z + (n-1)g$	2.354	/ 2.354	(x0.081)	(x0.081)	2.355	/ 2.355	(x0.089)	(x0.089)	3.268	/ 3.268	(x0.078)	(x0.078)
		1.447	/ 2.135	(x1.42)	(x1.38)	4.369	/ 7.739	(x1.40)	(x2.83)	13.36	/ 42.29	(x1.25)	(x5.72)
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	2.278	/ 2.278	(x0.08)	(x0.08)	2.375	/ 2.375	(x0.098)	(x0.098)	N/A			
		0.894	/ 2.191	(x1.62)	(x1.34)	2.843	/ 7.519	(x1.39)	(x2.59)				
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	2.24	/ 2.24	(x0.086)	(x0.086)	2.243	/ 2.243	(x0.077)	(x0.077)	N/A			
		0.445	/ 1.284	(x1.98)	(x0.637)	1.324	/ 2.227	(x1.42)	(x1.58)				
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	2.274	/ 2.274	(x0.08)	(x0.08)	2.123	/ 2.123	(x0.081)	(x0.081)	N/A			
		0.155	/ 1.258	(x3.38)	(x0.586)	0.602	/ 2.114	(x1.97)	(x1.62)				
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	2.436	/ 2.436	(x0.076)	(x0.076)	2.642	/ 2.642	(x0.082)	(x0.082)	N/A			
		2.162	/ 1.666	(x1.42)	(x0.592)	6.301	/ 3.359	(x1.22)	(x1.32)				
6	$gg \rightarrow t\bar{t} + (n-2)g$	3.541	/ 3.541	(x0.124)	(x0.124)	6.547	/ 6.547	(x0.131)	(x0.131)	27.57	/ 27.57	(N/A)	(N/A)
		2.42	/ 4.545	(x1.53)	(x3.49)	8.19	/ 26.36	(x1.89)	(x4.85)	22.29	/ 196	(N/A)	(N/A)
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	2.515	/ 2.515	(x0.137)	(x0.137)	3.016	/ 3.016	(x0.173)	(x0.173)	N/A			
		0.908	/ 2.564	(x1.66)	(x2.13)	2.985	/ 10.54	(x1.41)	(x3.06)				
8	$gg \rightarrow gg + (n-2)g$	2.404	/ 2.404	(x0.613)	(x0.613)	3.017	/ 3.017	(x6.05)	(x6.05)	N/A			
		3.293	/ 13.83	(x1.33)	(x1.92)	10.42	/ 103	(x1.66)	(x3.86)				
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	3.253	/ 3.253	(x0.061)	(x0.061)	4.959	/ 4.959	(x0.047)	(x0.047)	N/A			
		4.918	/ 1.568	(x1.25)	(x0.424)	13.26	/ 2.501	(x1.18)	(x0.747)				
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	4.138	/ 4.138	(x0.046)	(x0.046)	4.831	/ 4.831	(x0.041)	(x0.041)	N/A			
		0.772	/ 1.559	(x2.21)	(x0.429)	1.649	/ 1.762	(x1.86)	(x0.548)				
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	3.318	/ 3.318	(x0.067)	(x0.067)	5.426	/ 5.426	(x0.077)	(x0.077)	N/A			
		1.095	/ 1.621	(x1.72)	(x0.866)	3.436	/ 2.825	(x1.52)	(x1.51)				
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	4.26	/ 4.26	(x0.044)	(x0.044)	5.192	/ 5.192	(x0.037)	(x0.037)	N/A			
		1.495	/ 1.757	(x1.07)	(x0.309)	2.922	/ 2.064	(x0.966)	(x0.362)				
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	2.327	/ 2.327	(x0.075)	(x0.075)	3.437	/ 3.437	(x0.063)	(x0.063)	N/A			
		0.175		(x2.59)	(N/A)	0.673		(x1.44)	(N/A)				
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A				N/A				N/A			
summary: gen		2.24	4.26	2.87	2.44	2.12	6.55	3.7	3.02	3.27	27.6	15.4	15.4
		x0.044	x0.613	x0.121	x0.08	x0.037	x6.05	x0.542	x0.081	x0.455	x0.078	x0.078	x0.078
summary: run one-flow, hel.		0.155	4.92	1.55	1.09	0.602	13.3	4.54	2.98	13.4	22.3	17.8	17.8
sum		x1.07	x3.38	x1.78	x1.62	x0.966	x1.97	x1.49	x1.42	x1.45	x1.25	x1.25	x1.25
summary: run all-flows, one-hel		1.26	13.8	3	1.71	1.76	103	14.3	3.09	42.3	196	119	119
		x0.309	x3.49	x1.18	x0.751	x0.362	x4.85	x2.07	x1.60	x3.59	x5.72	x5.72	x5.72

Generic LC Performance Matrix (continued)

ID base process	n=7					n=8	n=9
1 $d\bar{d} \rightarrow Z + (n-1)g$	4.994	/ 4.994	(x0.08)	(x0.08)		N/A	N/A
	37.07	/ 309	(x1.52)	(x7.31)			
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A					N/A	N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A					N/A	N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A					N/A	N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A					N/A	N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A					N/A	N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A					N/A	N/A
8 $gg \rightarrow gg + (n-2)g$	N/A					N/A	N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A					N/A	N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A					N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A					N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A					N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A					N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A					N/A	N/A
summary: gen	4.99	4.99	4.99	4.99		N/A	N/A
	x0.08	x0.08	x0.08	x0.08	x0.08	N/A	N/A
summary: run one-flow, hel.	37.1	37.1	37.1	37.1		N/A	N/A
sum	x1.52	x1.52	x1.52	x1.52	x1.52	N/A	N/A
summary: run all-flows, one-hel	309	309	309	309		N/A	N/A
	x7.31	x7.31	x7.31	x7.31	x7.31	N/A	N/A

Matrix run settings. Generated by `docs/result_matrix.py` with `-color-accuracy lc`, from `result_matrix_data.json` into `result_matrix_table.tex`. AMPLICOL uses generated-library creation, direct timing of the first group/integral, and a fixed-helicity colour probe for all-flow timing. PYAMPLICOL JIT writes a pruned `selected_flow` artifact for the matched helicity-summed LC sector and an `all_flows -lc-topology-replay` artifact for the exact all-ordering fixed-helicity sum. The all-flow generation time reported in each cell is the generation time of that replay-partition artifact. PYAMPLICOL uses Rusticol double precision, batch size 64, output chunk size 128, stage-local evaluator inputs, ten Horner iterations, Symbolica's default CPE iteration choice, and enlarged Horner/common-pair limits. JIT rows use SymJIT O1; timing runs use `-target-runtime 10`. The LC matrix displays the selected-flow and all-flow JIT workloads side by side.

8 Dedicated $d\bar{d} \rightarrow Z + \text{Gluon}$ Performance

n	process	route	setup	selected flow, helicity sum				all flows, fixed helicity				notes
				gen [s]	wall [us/pt]	eval [us/pt]		gen [s]	wall [us/pt]	eval [us/pt]		
1	$d\bar{d} \rightarrow Z + (1-1)*g$	reference	AMPLICOL	2.47	N/A	0.0211		2.47	N/A	0.95		
1	$d\bar{d} \rightarrow Z + (1-1)*g$	Rusticol	PYAMPLICOL JIT O1	0.176 (x0.07)	0.179 (x8.50)	0.0466 (x2.21)	0.176 (x0.07)	0.156 (x0.16)	0.0331 (x0.03)			
1	$d\bar{d} \rightarrow Z + (1-1)*g$	Rusticol	PYAMPLICOL ASM O3	0.699 (x0.28)	0.224 (x10.6)	0.0906 (x4.29)	0.699 (x0.28)	0.154 (x0.16)	0.0313 (x0.03)			
1	$d\bar{d} \rightarrow Z + (1-1)*g$	Rusticol	PYAMPLICOL C++ O3	0.679 (x0.28)	0.172 (x8.17)	0.041 (x1.94)	0.679 (x0.28)	0.148 (x0.16)	0.0253 (x0.03)			
1	$d\bar{d} \rightarrow Z + (1-1)*g$	Rusticol	PYAMPLICOL JIT O3	0.174 (x0.07)	0.213 (x10.1)	0.0603 (x2.86)	0.174 (x0.07)	0.167 (x0.18)	0.0358 (x0.04)			
2	$d\bar{d} \rightarrow Z + (2-1)*g$	reference	AMPLICOL	2.94	N/A	0.128		2.94	N/A	1.08		
2	$d\bar{d} \rightarrow Z + (2-1)*g$	Rusticol	PYAMPLICOL JIT O1	0.191 (x0.06)	0.376 (x2.93)	0.195 (x1.52)	0.191 (x0.06)	0.214 (x0.20)	0.0789 (x0.07)			
2	$d\bar{d} \rightarrow Z + (2-1)*g$	Rusticol	PYAMPLICOL ASM O3	0.941 (x0.32)	0.727 (x5.67)	0.545 (x4.25)	0.941 (x0.32)	0.278 (x0.26)	0.142 (x0.13)			
2	$d\bar{d} \rightarrow Z + (2-1)*g$	Rusticol	PYAMPLICOL C++ O3	1.04 (x0.35)	0.409 (x3.19)	0.227 (x1.77)	1.04 (x0.35)	0.205 (x0.19)	0.0691 (x0.06)			
2	$d\bar{d} \rightarrow Z + (2-1)*g$	Rusticol	PYAMPLICOL JIT O3	0.273 (x0.09)	0.406 (x3.16)	0.209 (x1.63)	0.273 (x0.09)	0.224 (x0.21)	0.0829 (x0.08)			
3	$d\bar{d} \rightarrow Z + (3-1)*g$	reference	AMPLICOL	2.15	N/A	0.413		2.15	N/A	1.18		
3	$d\bar{d} \rightarrow Z + (3-1)*g$	Rusticol	PYAMPLICOL JIT O1	0.205 (x0.10)	0.803 (x1.94)	0.537 (x1.30)	0.205 (x0.10)	0.591 (x0.50)	0.329 (x0.28)			
3	$d\bar{d} \rightarrow Z + (3-1)*g$	Rusticol	PYAMPLICOL ASM O3	1.21 (x0.57)	1.94 (x4.70)	1.67 (x4.04)	1.21 (x0.57)	0.977 (x0.83)	0.718 (x0.61)			
3	$d\bar{d} \rightarrow Z + (3-1)*g$	Rusticol	PYAMPLICOL C++ O3	1.35 (x0.63)	0.951 (x2.30)	0.684 (x1.66)	1.35 (x0.63)	0.579 (x0.49)	0.319 (x0.27)			
3	$d\bar{d} \rightarrow Z + (3-1)*g$	Rusticol	PYAMPLICOL JIT O3	0.176 (x0.08)	0.798 (x1.93)	0.533 (x1.29)	0.176 (x0.08)	0.584 (x0.50)	0.325 (x0.28)			
4	$d\bar{d} \rightarrow Z + (4-1)*g$	reference	AMPLICOL	2.35	N/A	1.45		2.35	N/A	2.13		
4	$d\bar{d} \rightarrow Z + (4-1)*g$	Rusticol	PYAMPLICOL JIT O1	0.218 (x0.09)	1.97 (x1.36)	1.53 (x1.06)	0.218 (x0.09)	2.87 (x1.35)	1.99 (x0.93)			
4	$d\bar{d} \rightarrow Z + (4-1)*g$	Rusticol	PYAMPLICOL ASM O3	1.58 (x0.67)	5.47 (x3.78)	5.01 (x3.46)	1.58 (x0.67)	5.92 (x2.77)	5.01 (x2.35)			
4	$d\bar{d} \rightarrow Z + (4-1)*g$	Rusticol	PYAMPLICOL C++ O3	1.81 (x0.77)	2.7 (x1.87)	2.24 (x1.55)	1.81 (x0.77)	2.98 (x1.40)	2.09 (x0.98)			
4	$d\bar{d} \rightarrow Z + (4-1)*g$	Rusticol	PYAMPLICOL JIT O3	0.191 (x0.08)	2.05 (x1.42)	1.59 (x1.10)	0.191 (x0.08)	2.95 (x1.38)	2.05 (x0.96)			
5	$d\bar{d} \rightarrow Z + (5-1)*g$	reference	AMPLICOL	2.36	N/A	4.37		2.36	N/A	7.74		
5	$d\bar{d} \rightarrow Z + (5-1)*g$	Rusticol	PYAMPLICOL JIT O1	0.252 (x0.11)	6.21 (x1.42)	5.19 (x1.19)	0.252 (x0.11)	21.6 (x2.79)	15.6 (x2.02)			
5	$d\bar{d} \rightarrow Z + (5-1)*g$	Rusticol	PYAMPLICOL ASM O3	1.85 (x0.79)	16.7 (x3.83)	15.6 (x3.57)	1.85 (x0.79)	49.2 (x6.36)	42.2 (x5.46)			
5	$d\bar{d} \rightarrow Z + (5-1)*g$	Rusticol	PYAMPLICOL C++ O3	2.5 (x1.06)	8.33 (x1.91)	7.27 (x1.66)	2.5 (x1.06)	23.6 (x3.05)	17.6 (x2.27)			
5	$d\bar{d} \rightarrow Z + (5-1)*g$	Rusticol	PYAMPLICOL JIT O3	0.21 (x0.09)	6.1 (x1.40)	5.11 (x1.17)	0.21 (x0.09)	21.9 (x2.83)	15.7 (x2.03)			
6	$d\bar{d} \rightarrow Z + (6-1)*g$	reference	AMPLICOL	3.27	N/A	13.4		3.27	N/A	42.3		
6	$d\bar{d} \rightarrow Z + (6-1)*g$	Rusticol	PYAMPLICOL JIT O1	0.279 (x0.09)	16.4 (x1.23)	14.7 (x1.10)	0.279 (x0.09)	241 (x5.70)	168 (x3.97)			
6	$d\bar{d} \rightarrow Z + (6-1)*g$	Rusticol	PYAMPLICOL ASM O3	2.16 (x0.66)	47.4 (x3.55)	44.9 (x3.36)	2.16 (x0.66)	480 (x11.4)	407 (x9.61)			
6	$d\bar{d} \rightarrow Z + (6-1)*g$	Rusticol	PYAMPLICOL C++ O3	N/A	N/A	N/A	N/A	N/A	N/A			
6	$d\bar{d} \rightarrow Z + (6-1)*g$	Rusticol	PYAMPLICOL JIT O3	0.255 (x0.08)	16.7 (x1.25)	14.9 (x1.12)	0.255 (x0.08)	242 (x5.72)	168 (x3.98)			
7	$d\bar{d} \rightarrow Z + (7-1)*g$	reference	AMPLICOL	N/A	N/A	N/A	N/A	N/A	N/A	N/A		
7	$d\bar{d} \rightarrow Z + (7-1)*g$	Rusticol	PYAMPLICOL JIT O1	N/A	N/A	N/A	N/A	N/A	N/A	N/A		
7	$d\bar{d} \rightarrow Z + (7-1)*g$	Rusticol	PYAMPLICOL ASM O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A		
7	$d\bar{d} \rightarrow Z + (7-1)*g$	Rusticol	PYAMPLICOL C++ O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A		
7	$d\bar{d} \rightarrow Z + (7-1)*g$	Rusticol	PYAMPLICOL JIT O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A		
8	$d\bar{d} \rightarrow Z + (8-1)*g$	reference	AMPLICOL	N/A	N/A	N/A	N/A	N/A	N/A	N/A		
8	$d\bar{d} \rightarrow Z + (8-1)*g$	Rusticol	PYAMPLICOL JIT O1	N/A	N/A	N/A	N/A	N/A	N/A	N/A		
8	$d\bar{d} \rightarrow Z + (8-1)*g$	Rusticol	PYAMPLICOL ASM O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A		
8	$d\bar{d} \rightarrow Z + (8-1)*g$	Rusticol	PYAMPLICOL C++ O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A		
8	$d\bar{d} \rightarrow Z + (8-1)*g$	Rusticol	PYAMPLICOL JIT O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A		
9	$d\bar{d} \rightarrow Z + (9-1)*g$	reference	AMPLICOL	N/A	N/A	N/A	N/A	N/A	N/A	N/A		
9	$d\bar{d} \rightarrow Z + (9-1)*g$	Rusticol	PYAMPLICOL JIT O1	N/A	N/A	N/A	N/A	N/A	N/A	N/A		
9	$d\bar{d} \rightarrow Z + (9-1)*g$	Rusticol	PYAMPLICOL ASM O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A		
9	$d\bar{d} \rightarrow Z + (9-1)*g$	Rusticol	PYAMPLICOL C++ O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A		
9	$d\bar{d} \rightarrow Z + (9-1)*g$	Rusticol	PYAMPLICOL JIT O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A		

Final-state multiplicity n means $d\bar{d} \rightarrow Z + (n-1)g$. PYAMPLICOL rows use Rusticol, batch size 64, output chunk size 128, stage-local evaluator inputs, ten Horner iterations, and `time-process -target-runtime 10`. The first timing block is one selected LC flow with the helicity sum; the second is the sum over all LC flows for one fixed source helicity. For PYAMPLICOL rows, generation columns report all-flow artifact generation. The green row is the default PYAMPLICOL route, SymJIT O3. Generated processes are kept under `docs/.z_performance_outputs`; the driver is `docs/run_z_performance_table.py`.

To reproduce one PYAMPLICOL row, run generation followed by timing. This example is the $n = 7$ JIT O3 selected-flow block; for the all-flow block, use the same command but replace the sector/order flags with `-lc-topology-replay -skip-generic-plan -no-runtime-lc-sector-selector -source-helicities 1=-1,2=-1,4=1,5=1,6=1,7=1,8=1,9=1`.

```
env PYTHONPATH=src dependencies/.venv/bin/python -m pyamplicol generate-process 'd d~ > z g g g g g' docs/.z_performance_outputs/manual/n7/jit_o3 --replace --n_cores 5 --color-accuracy lc --batch-size 64 --symbolica-n_cores 5
--symbolica-output-chunk-size 128 --symbolica-stage-local-parameter-layout --symbolica-iterations 10 --symbolica-max-horner-scheme-variables 1000 --symbolica-max-common-pair-cache-entries 5000000 --symbolica-max-common-pair-distance 1000
--lc-sector-ids 0 --reference-color-order 2,4,5,6,7,8,9,1,3 --symbolica-evaluator-backend jit --symbolica-jit-optimization-level 3
env PYTHONPATH=src dependencies/.venv/bin/python -m pyamplicol time-process --target-runtime 10 --batch-size 64 --json docs/.z_performance_outputs/manual/n7/jit_o3
```

9 Generic NLC Performance Matrix

Cell format: generation time above runtime per phase-space point; slots are AMPLICOL, PYAMPLICOL JIT O1, and PYAMPLICOL C++ O3. AMPLICOL uses raw generated-library colour probes. Conventions and gaps are summarized after the table.

ID	base process	n=1	n=2	n=3
1	$d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A	N/A
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A	N/A
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A	N/A
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A	N/A
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A	N/A
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
8	$gg \rightarrow gg + (n-2)g$	N/A	N/A	N/A
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: gen		N/A	N/A	N/A
		N/A	N/A	N/A
summary: run one-flow, hel.		N/A	N/A	N/A
sum		N/A	N/A	N/A

Generic NLC Performance Matrix (continued)

ID	base process	n=4	n=5	n=6
1	$d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A	N/A
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A	N/A
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A	N/A
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A	N/A
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A	N/A
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
8	$gg \rightarrow gg + (n-2)g$	N/A	N/A	N/A
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: gen		N/A	N/A	N/A
		N/A	N/A	N/A
summary: run one-flow, hel.		N/A	N/A	N/A
sum		N/A	N/A	N/A

Each cell compares raw generated-library AMPLICOL colour probes against PYAMPLICOL staged-DAG JIT O1 and C++ O3 at the same final-state multiplicity n . Cell slots are AMPLICOL, PYAMPLICOL JIT, and PYAMPLICOL C++ O3; each slot shows generation time above runtime per phase-space point. PYAMPLICOL runtime multipliers are wall-time ratios relative to AMPLICOL. Generation entries are in seconds; AMPLICOL runtime entries and absolute PYAMPLICOL runtimes are in microseconds per point. If no direct AMPLICOL reference exists, completed PYAMPLICOL slots show absolute timings. For NLC/full-colour tables, AMPLICOL reference values come from the dedicated raw generated-library colour driver `amplicol_color_library_probe`. Ratio colours are green below one, orange below two, and red otherwise. The summary rows give `min|max|avg|med`; multiplier summaries add `sum`, the ratio of paired summed PYAMPLICOL and AMPLICOL times. For LC, the two runtime summary rows match the one-flow helicity-summed and all-flow fixed-helicity runtime slots. Missing and structural N/A cells are ignored. Validated rows use the same phase-space point and a 10^{-8} relative tolerance.

A red **VALIDATION FAILED** marker is shown next to any validated entry whose relative difference exceeds 10^{-8} .

Matrix run settings. Generated by `docs/result_matrix.py` with `-color-accuracy nlc`, from `result_matrix_nlc_data.json` into `result_matrix_nlc_table.tex`. AMPLICOL uses the raw generated-library path `amplicol_generate -library=create-raw` followed by `amplicol_color_library_probe`, preserving NLC/full colour-order interferences. PYAMPLICOL uses Rusticol double precision, batch size 64, output chunk size 128, stage-local evaluator inputs, ten Horner iterations, Symbolica's default CPE iteration choice, and enlarged Horner/common-pair limits. JIT rows use SymJIT O1; timing runs use `-target-runtime 10`. Only C++ O3 generation is subject to the 15-minute compile cap.

10 Generic Full-Colour Performance Matrix

Cell format: generation time above runtime per phase-space point; slots are AMPLICOL, PYAMPLICOL JIT O1, and PYAMPLICOL C++ O3. AMPLICOL uses raw generated-library colour probes. Conventions and gaps are summarized after the table.

ID	base process	n=1	n=2	n=3
1	$d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A	N/A
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A	N/A
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A	N/A
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A	N/A
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A	N/A
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
8	$gg \rightarrow gg + (n-2)g$	N/A	N/A	N/A
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: gen		N/A	N/A	N/A
		N/A	N/A	N/A
summary: run one-flow, hel.		N/A	N/A	N/A
sum		N/A	N/A	N/A

Generic Full-Colour Performance Matrix (continued)

ID base process	n=4	n=5	n=6
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A	N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A	N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A	N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A	N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A	N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A	N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: gen	N/A	N/A	N/A
	N/A	N/A	N/A
summary: run one-flow, hel.	N/A	N/A	N/A
sum	N/A	N/A	N/A

Each cell compares raw generated-library AMPLICOL colour probes against PYAMPLICOL staged-DAG JIT O1 and C++ O3 at the same final-state multiplicity n . Cell slots are AMPLICOL, PYAMPLICOL JIT, and PYAMPLICOL C++ O3; each slot shows generation time above runtime per phase-space point. PYAMPLICOL runtime multipliers are wall-time ratios relative to AMPLICOL. Generation entries are in seconds; AMPLICOL runtime entries and absolute PYAMPLICOL runtimes are in microseconds per point. If no direct AMPLICOL reference exists, completed PYAMPLICOL slots show absolute timings. For NLC/full-colour tables, AMPLICOL reference values come from the dedicated raw generated-library colour driver `amplicol_color_library_probe`. Ratio colours are green below one, orange below two, and red otherwise. The summary rows give `min|max|avg|med`; multiplier summaries add `sum`, the ratio of paired summed PYAMPLICOL and AMPLICOL times. For LC, the two runtime summary rows match the one-flow helicity-summed and all-flow fixed-helicity runtime slots. Missing and structural N/A cells are ignored. Validated rows use the same phase-space point and a 10^{-8} relative tolerance.

A red **VALIDATION FAILED** marker is shown next to any validated entry whose relative difference exceeds 10^{-8} .

Matrix run settings. Generated by `docs/result_matrix.py` with `-color-accuracy full`, from `result_matrix_full_data.json` into `result_matrix_full_table.tex`. AMPLICOL uses the raw generated-library path `amplicol_generate -library=create-raw` followed by `amplicol_color_library_probe`, preserving NLC/full colour-order interferences. PYAMPLICOL uses Rusticol double precision, batch size 64, output chunk size 128, stage-local evaluator inputs, ten Horner iterations, Symbolica’s default CPE iteration choice, and enlarged Horner/common-pair limits. JIT rows use SymJIT O1; timing runs use `-target-runtime 10`. Only C++ O3 generation is subject to the 15-minute compile cap.

Archived low-multiplicity validation fixture. The $n \leq 4$ LC/NLC/full-colour validation cells are mirrored in `tests/fixtures/result_matrix_references.json`. The integration test checks those cached same-point AMPLICOL/PYAMPLICOL values without a local Fortran run; `scripts/refresh_result_matrix_fixtures.py` refreshes the fixture from the matrix caches.

11 Why This Matches AmpliCol but Remains Python-Friendly

The Python layer owns graph construction, process metadata, deterministic phase-space steering, artifact generation, evaluator caching, and validation against AMPLICOL. It does not perform the hot current recursion component by component in production. The two production execution routes are now:

Python staged route: source fill $\rightarrow$ bounded stage evaluators $\rightarrow$ amplitude/raw-sum evaluator,

Rusticol route: one PyO3 call $\rightarrow$ Rust source fill and staged sweep $\rightarrow$ amplitude/raw-sum evaluator.

Thus PYAMPLICOL keeps AMPLICOL’s essential optimization - shared currents across helicity parents and a topological current-table sweep - while replacing generated Fortran subroutines by saved Spensio/Symbolica evaluator artifacts. The `rusticol` path removes the remaining Python orchestration cost by loading the same process directory and executing the sweep in Rust.

The old fully monolithic compiled-DAG route is now deprecated and kept only as reference code. Its alias-based design was useful for exploring whole-graph compilation, but the production target is the staged DAG: Python generation plus a Rust eager-DAG runtime over saved Symbolica stage evaluators. This keeps the runtime schedule close to AMPLICOL’s current-table library while leaving the graph compiler model-driven and process-generic.